Module 12 - chapter 23
Complete CI/CD with Terraform - Part 2

The installation documentation for Terraform can be found here: https://developer.hashicorp.com/terraform/downloads

The project files used in this lecture can be found here:
- Terraform Learn: https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn/-/tree/feature/eks
- Java-maven-app: https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/java-maven-app

The installation documentation for Docker Compose standalone can be found here: https://docs.docker.com/compose/install/standalone/

Create a new multi-branch pipeline

I used my-multi-branch-pipeline as reference to create terraform-pipeline and also I review Module 8 - chapter 15
Webhooks - Trigger Pipeline Jobs automatically to add the webhook credential to the new pipeline. my-multi-branch-pipeline has a credential called my-pipeline which is use

After I created the new multi-branch pipeline Jenkins registered the webhook in the repo automatically (my terraform repo didn't have a webhook before this)

— tangent —
Your Jenkins server is configured to automatically register a webhook on new GitHub repos when it detects them or when a new pipeline/job is created. This is standard Jenkins + GitHub integration behavior — Jenkins calls the GitHub API to create the webhook itself so it gets notified of pushes, PRs, etc.

So the flow is: you create a new repo → Jenkins notices (or you create a job pointing at it) → Jenkins automatically calls the GitHub API to register its webhook on that repo.

You don't need to create webhooks manually — Jenkins is handling it for you. The GitHub CLI OAuth app is unrelated, that's just the gh command line tool.

Q:
did this happen when I created the multi branch pipeline? what I mean is that probably the repo didn't have the webhook and then I created the multibranch pipeline where I input the repo address and the jenkins register the webhook in the repo?

A:
Yes, exactly. That's the sequence:

1. New repo created — no webhook
2. You created the Multibranch Pipeline in Jenkins and entered the repo URL
3. Jenkins called the GitHub API to register its webhook on that repo automatically

The Multibranch Pipeline needs to know about new branches and PRs in real time, so it registers the webhook as part of the setup so GitHub can notify it of events. You don't have to do it manually because Jenkins does it for you at job creation time.

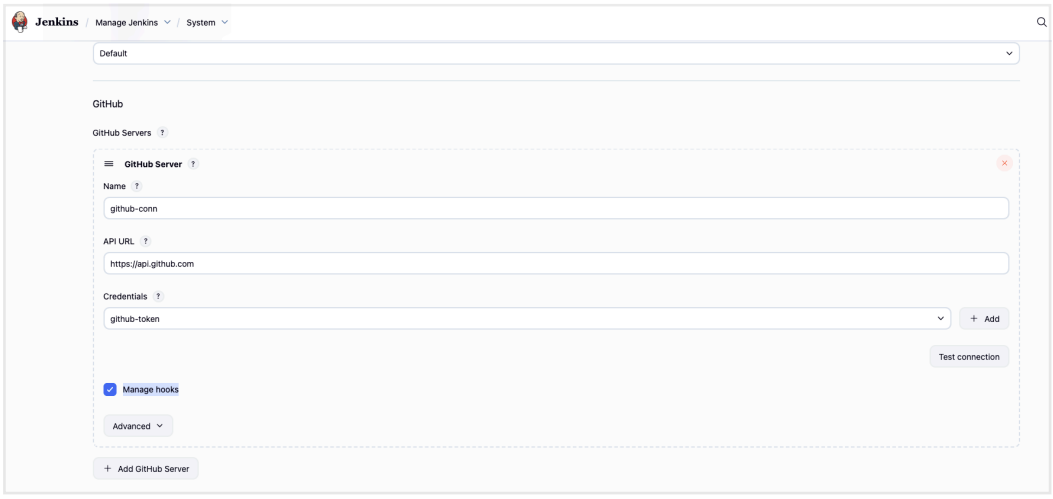
Q:
 where is the configuration of my jenkins that tells to do the registry automcatically?

A:
In your Jenkins UI, go to:

Manage Jenkins → System and look for the GitHub section. There you'll find a GitHub Server configuration with credentials (a Personal Access Token). When this is configured, Jenkins has the permission and knows to auto-register webhooks.

The specific setting is usually called "Manage hooks" — it's a checkbox next to the GitHub server entry. If that's checked, Jenkins will automatically create/manage webhooks on repos whenever a job is pointed at them.

That's the setting that enables the behavior you're seeing.



— end tangent —

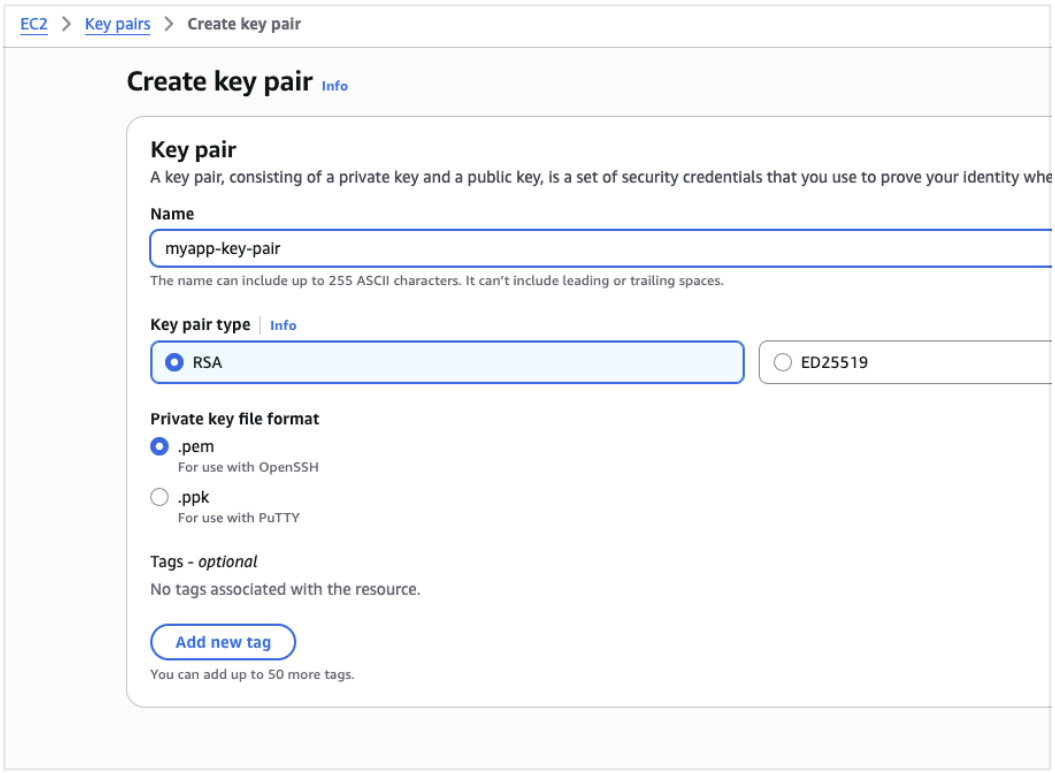
Create the ssh key pair

In module 12 - chapter 13 we created a key pair via terraform in our laptops

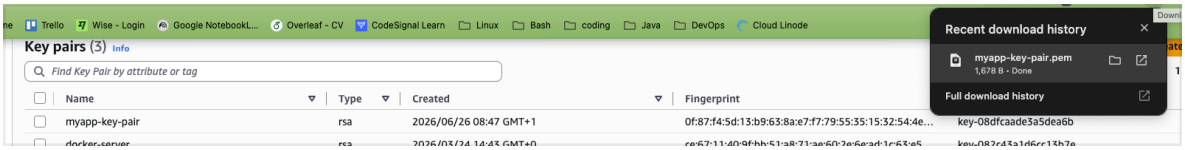
```
resource "aws_key_pair" "ssh-key" {
  key_name   = "server-key"
  public_key = file("~/.ssh/id_rsa.pub")
}
```

And also we can do it using AWS console.

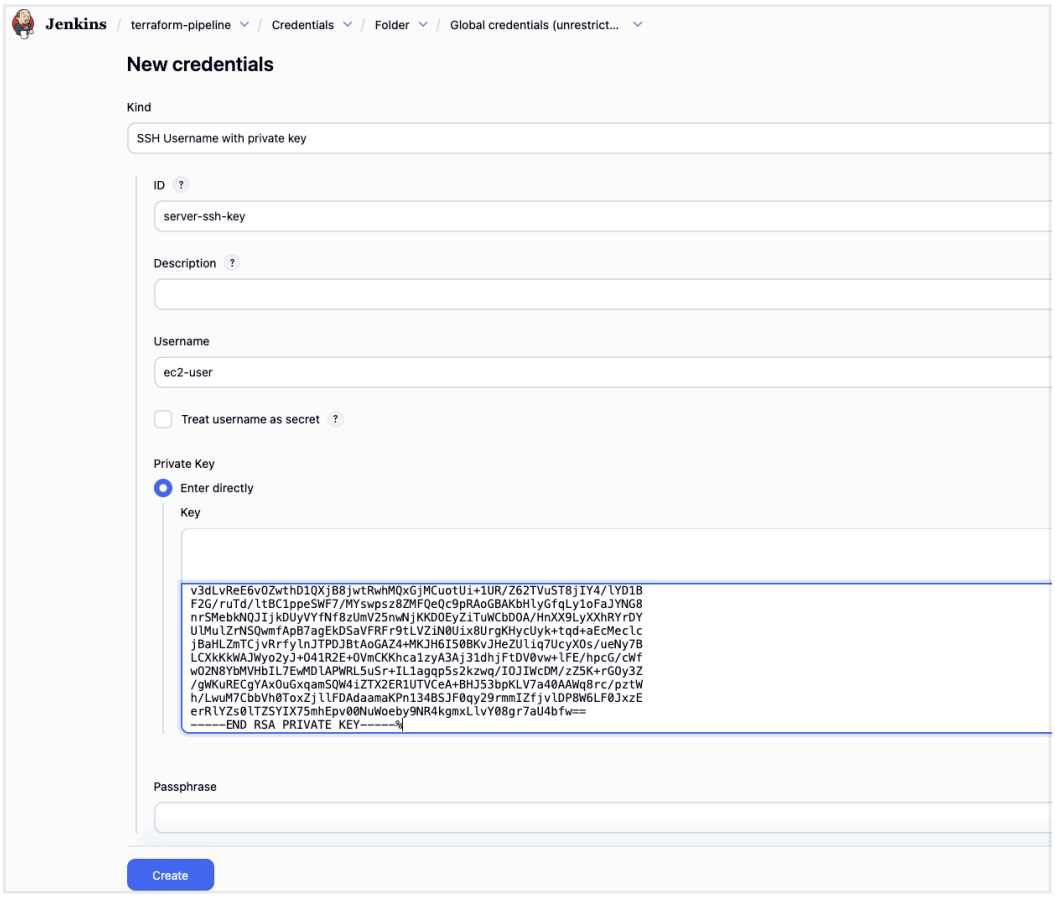
For this demo we will use AWS to create the key pair (instead of using tf) and then we will pass it to jenkins container.



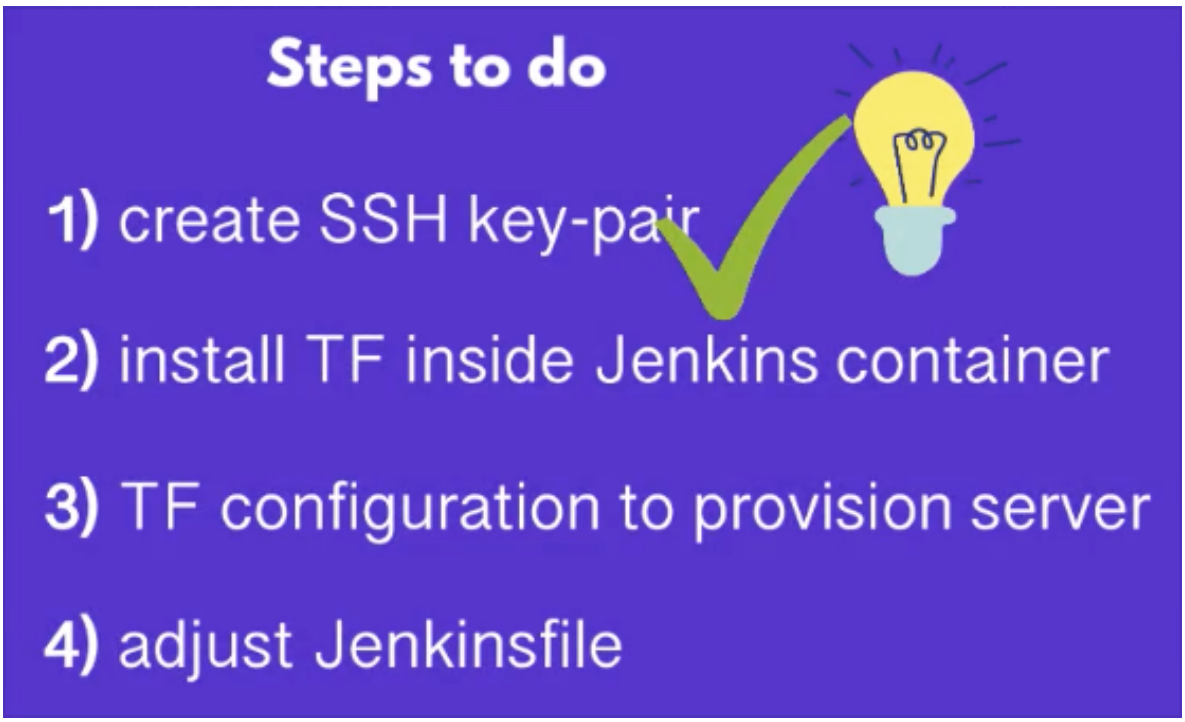
And the key-pair pem file has been downloaded automatically upon creation in AWS



Now we create a new SSH credential in Jenkins using the contents of the pem file:



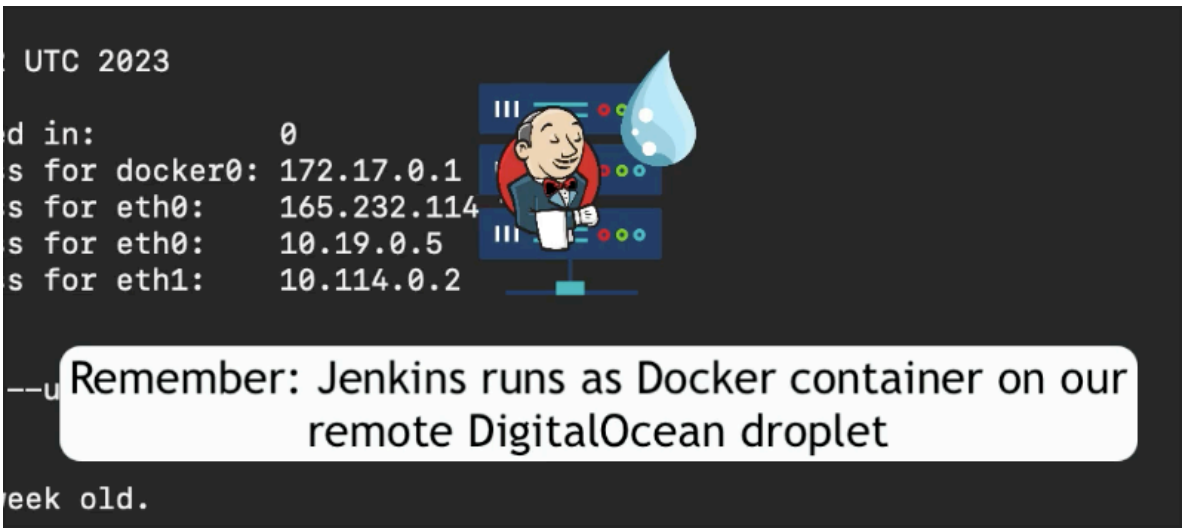
First step is done



Now jenkins will be able to ssh into the EC2 server to deploy the application using this private key.

Install terraform inside jenkins conatiner

I ssh into the droplet
-> ssh root@165.22.116.21



-> docker ps

We can see the container running jenkins and now we enter jenkins conatiner as user root:
-> docker exec -it -u 0 56efcedae10e bash

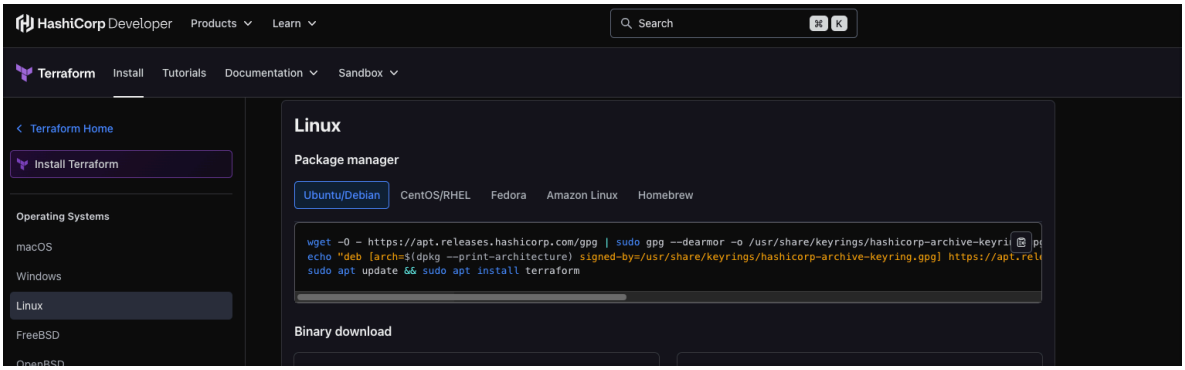
-u 0 -> user zero = user root -> super user

We check the OS of the container
-> cat etc/os-release

```
root@56efcedae10e:/# cat etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.1
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
root@56efcedae10e:/#
```

And we have a Debian distribution -> linux

Then we follow the installation instructions based on the OS we have -> https://developer.hashicorp.com/terraform/install



First command will add the HashiCorp key and write it to a file on our Jenkins Server
-> wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

Error:

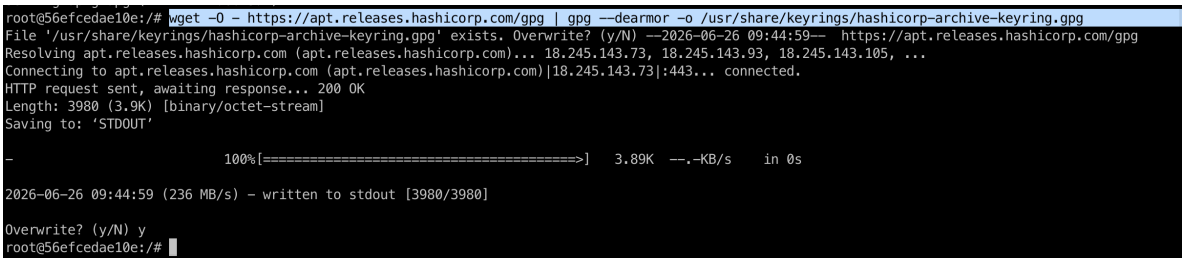
```
root@56efcedae10e:/# wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
bash: wget: command not found
bash: sudo: command not found
```

Problem:
You're running as root inside the container already (that's why sudo isn't found), and wget isn't installed. Try this instead:

-> apt-get update && apt-get install -y wget gpg

Then re-run the original command but without sudo.

-> wget -O - https://apt.releases.hashicorp.com/gpg | gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg



Next command adds the official hashicorp repository to the list of installation sources
installation sources (also called package sources or apt sources) — the list of places where apt looks to download software.

-> iecho "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | tee /etc/apt/sources.list.d/hashicorp.list

AI says:
The command you have adds HashiCorp's repository as a new source. The main sources on a Debian/Ubuntu system are:

- /etc/apt/sources.list — the main file, usually contains the OS's official repos (Ubuntu, Debian mirrors)
- /etc/apt/sources.list.d/ — a directory for additional sources, one file per third-party repo. This is where the command is writing hashicorp.list

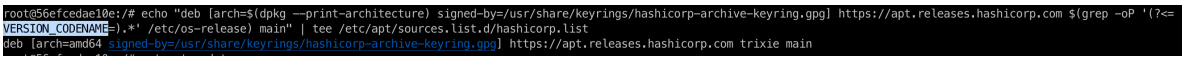
So when you run apt-get install terraform, apt knows to look in HashiCorp's repo because of that file.

Common examples of third-party sources added this way:
- HashiCorp (Terraform, Vault, Consul)
- Docker
- Google (kubectl, Chrome)
- NodeSource (Node.js)
- Jenkins itself

The GPG key you added earlier is the trust mechanism — apt uses it to verify that packages from that source actually came from HashiCorp and weren't tampered with.

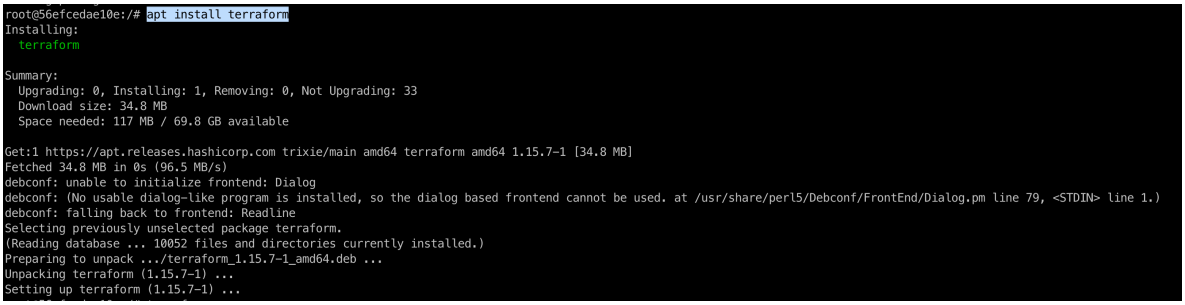
I am using Debian trixie (see -> cat etc/os-release) not ubuntu so I need to edit the command:

-> echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=**VERSION_CODENAME**=).*' /etc/os-release) main" | tee /etc/apt/sources.list.d/hashicorp.list

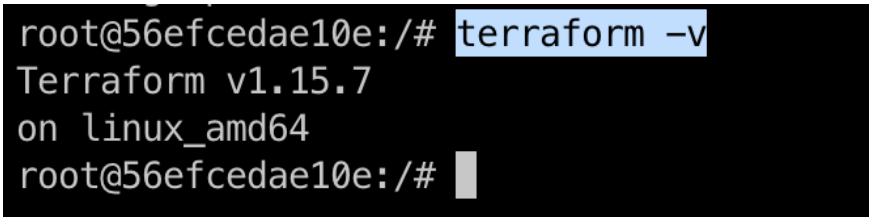


Next we install terraform
-> apt update

-> apt install terraform



-> terraform -v



— tangent —.

An alternative to install terraform and skip **apt** entirely is to just download the binary directly:

apt-get update && apt-get install -y curl unzip

curl -fsSL https://releases.hashicorp.com/terraform/1.11.4/

terraform_1.11.4_linux_amd64.zip -o terraform.zip && \
unzip terraform.zip && \
mv terraform /usr/local/bin/ && \
rm terraform.zip

Then verify:
terraform --version

This works regardless of the OS/distro and is actually the approach most people use in containers anyway.

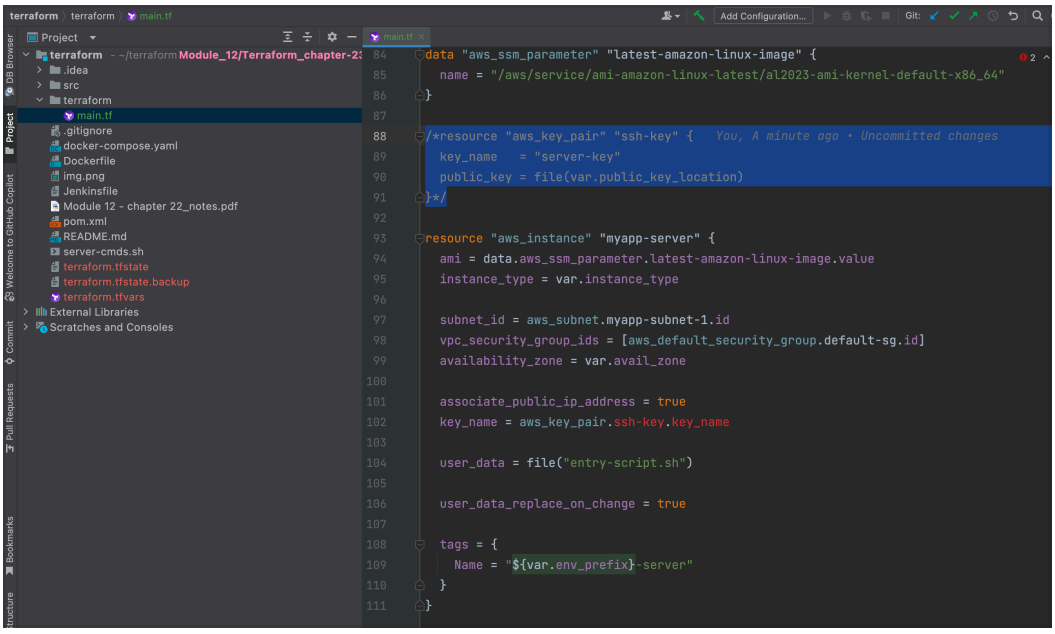
—- end tangent —

Terraform configuration files

We will add the tf config files in our application code

I create a new folder called ‘terraform’ and inside I create a file called ‘main.tf’ and I copy the code from Module 12 - chapter 14 ‘main,tf’ and paste it in the new file

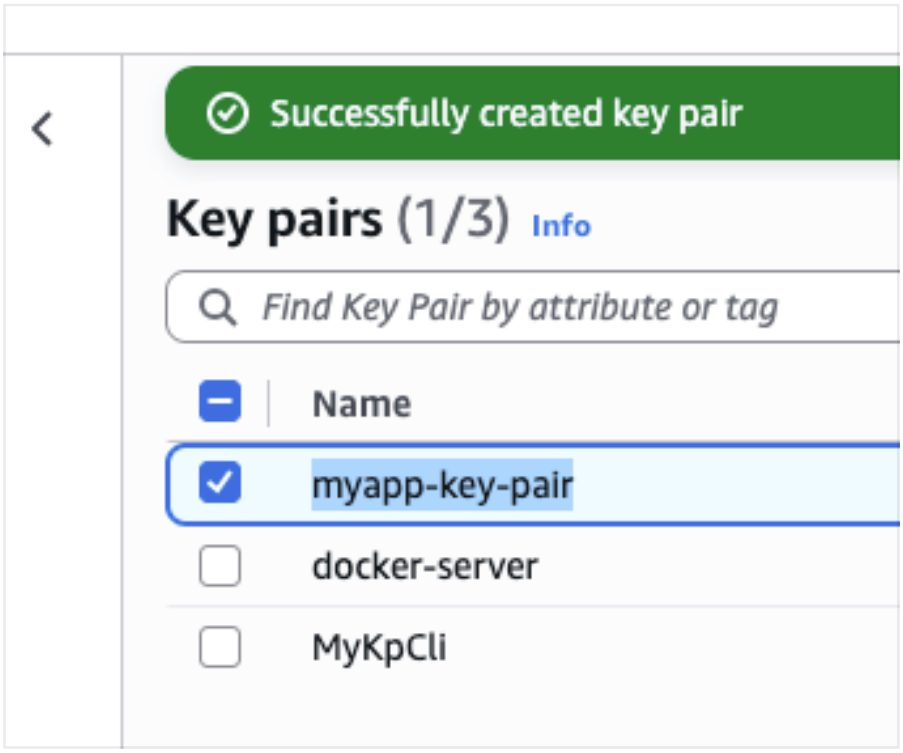
And we remove the key pair (line 88) resource from the code (we already did this part manually using AWS)



And we update line 97 👇

```
87
88  resource "aws_instance" "myapp-server" {
89    ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
90    instance_type = var.instance_type
91
92    subnet_id = aws_subnet.myapp-subnet-1.id
93    vpc_security_group_ids = [aws_default_security_group.default-sg.id]
94    availability_zone = var.avail_zone
95
96    associate_public_ip_address = true
97    key_name = aws_key_pair.ssh-key.key_name
98
99    user_data = file("entry-script.sh")
100
101   user_data_replace_on_change = true
102
103   tags = {
104     Name = "${var.env_prefix}-server"
105   }
106 }   You, A minute ago • Uncommitted changes
```

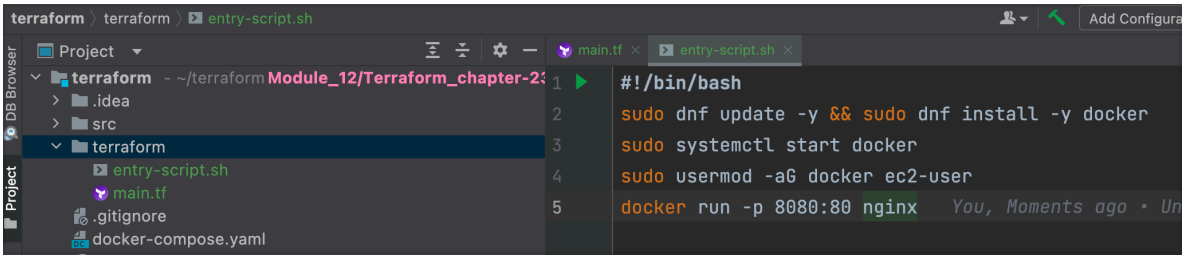
to use the AWS key that we created earlier -> myapp-key-pair



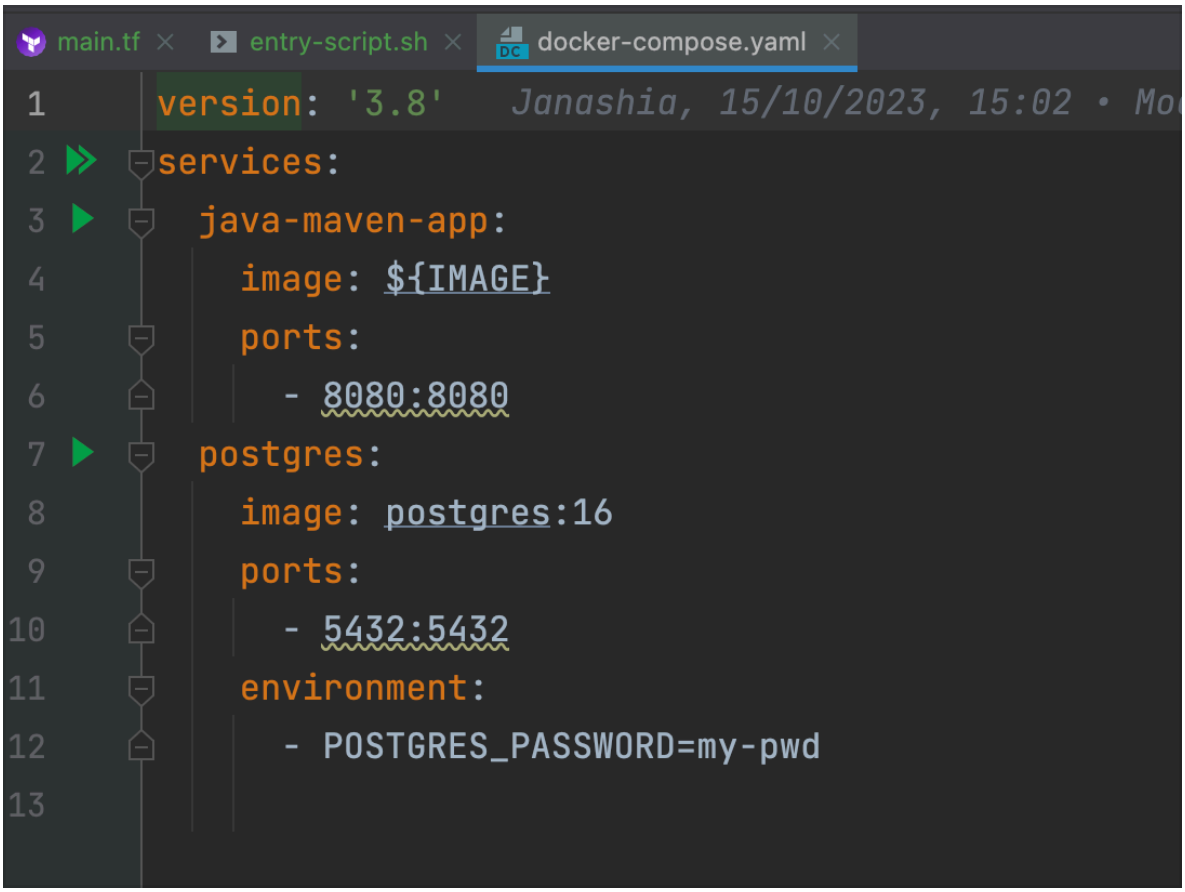
And now our code looks like this

```
95
96      associate_public_ip_address = true
97      key_name = "myapp-key-pair"
98
```

In line 99 we call ‘entry-script.sh; so we need that file in our project. I copy it and paste it



But we want to deploy and execute docker compose which will run two containers one for our app and another for the Postgres DB instead of getting docker to run nginx container.

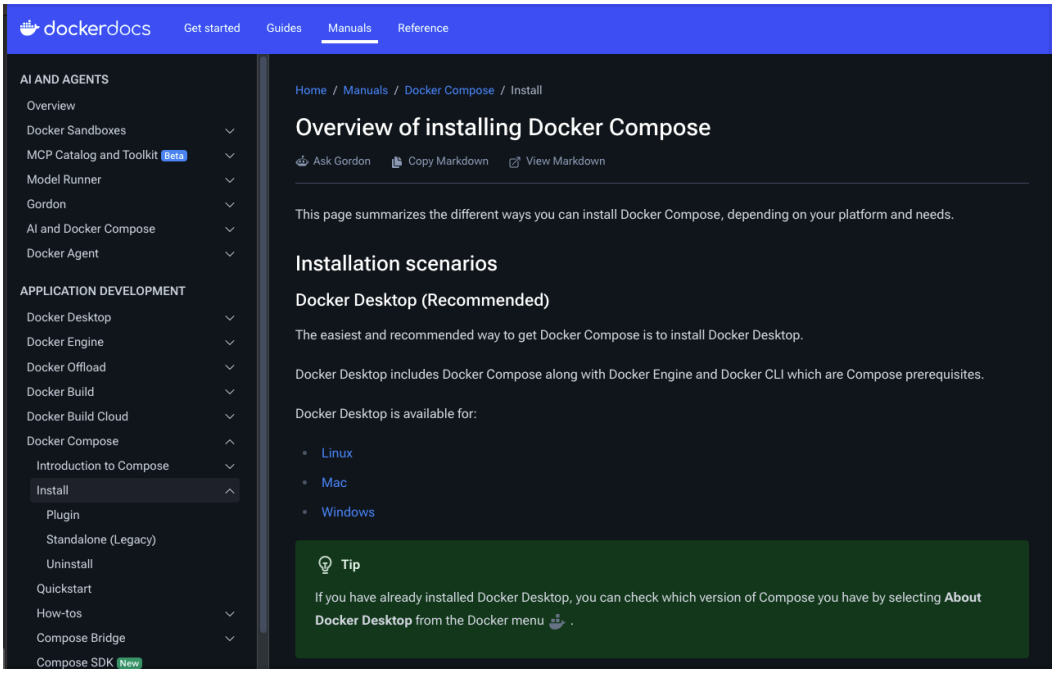


We will copy the docker-compose.yaml file into the EC2 instance and get docker-compose to run all the containers we need via sshagent (see lines 49-52)

```
Jenkinsfile M   main.tf U   $ entry-script.sh U
30            echo 'building docker image...'
31            buildImage(env.IMAGE_NAME)
32            dockerLogin()
33            dockerPush(env.IMAGE_NAME)
34          }
35        }
36      }
37      stage("provision server") {
38        // tf provision server
39        sh "terraform init"
40      }
41      stage("deploy") {
42        steps {
43          script {
44            echo 'deploying docker image to EC2...'
45
46            def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
47            def ec2Instance = "ec2-user@35.180.251.121"
48
49            sshagent(['ec2-server-key']) {
50              sh "scp server-cmds.sh ${ec2Instance}:/home/ec2-user"
51              sh "scp docker-compose.yaml ${ec2Instance}:/home/ec2-user"
52              sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
53            }
54          }
55        }
56      }
```

So we need to install docker compose as well.

https://docs.docker.com/compose/install



— tangent ——

In Module 11 - chapter 9 I created the jenkins conatiner using:

-> docker run -p 8080:8080 \
  -p 50000:50000 -d \
  -v /root/.kube:/var/jenkins_home/.kube \
  -v /usr/local/bin/kubectl:/usr/local/bin/kubectl \
  -v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  **-v /usr/bin/docker:/usr/bin/docker **
  jenkins/jenkins:lts

Q:
I need to write the code to install docker compose in my docker-compose.yaml the teacher is using docker-compose stand alone but the current documentation says that the standalone is legacy.

A:
You have two options:

Option 1: Same pattern, mount the binary (quickest)
Download docker-compose on the host and mount it in, just like you did with docker:
on the host
curl -SL https://github.com/docker/compose/releases/download/v2.36.2/docker-compose-linux-x86_64 -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose
Then add to your docker run:
-v /usr/local/bin/docker-compose:/usr/local/bin/docker-compose

Option 2: Create a Jenkins Dockerfile (cleaner, recommended)
Create a Dockerfile that extends the official image:
FROM jenkins/jenkins:lts

USER root
RUN apt-get update && apt-get install -y docker-compose-plugin
USER jenkins
Then build and run from that image instead of jenkins/jenkins:lts.

Option 2 is better long-term — everything Jenkins needs is self-contained in the image rather than depending on what's installed on the host. But Option 1 is faster if you just need it working now.

Q:
why I need this step: Then add to your docker run:
-v /usr/local/bin/docker-compose:/usr/local/bin/docker-compose

A:

Because the binary is installed on the host, not inside the container. Without mounting it, the Jenkins container has no idea it exists.

It's the same reason you mounted /usr/bin/docker — Docker was installed on the host and you needed to make it accessible inside the container by mounting it in.

Q:
so if for the demo I don't mounte it it will work but if the conatiner dies I loose everything

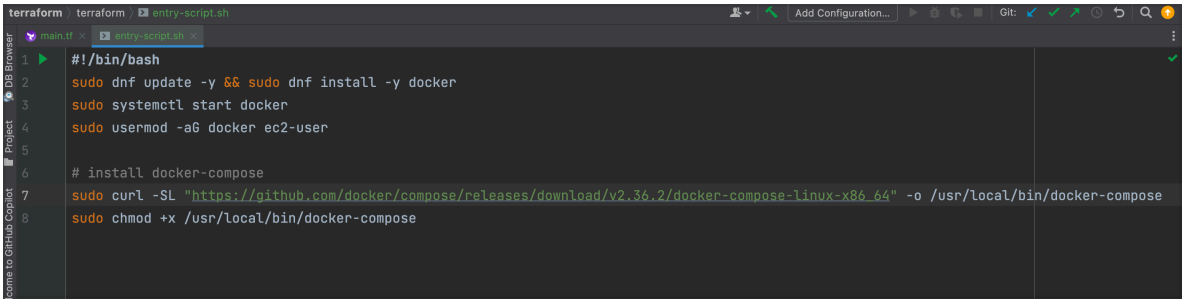
A:
Exactly. If you don't mount it:

- It works for the demo (you installed it manually inside the running container)
- Container dies → everything gone, you have to SSH in and install it again

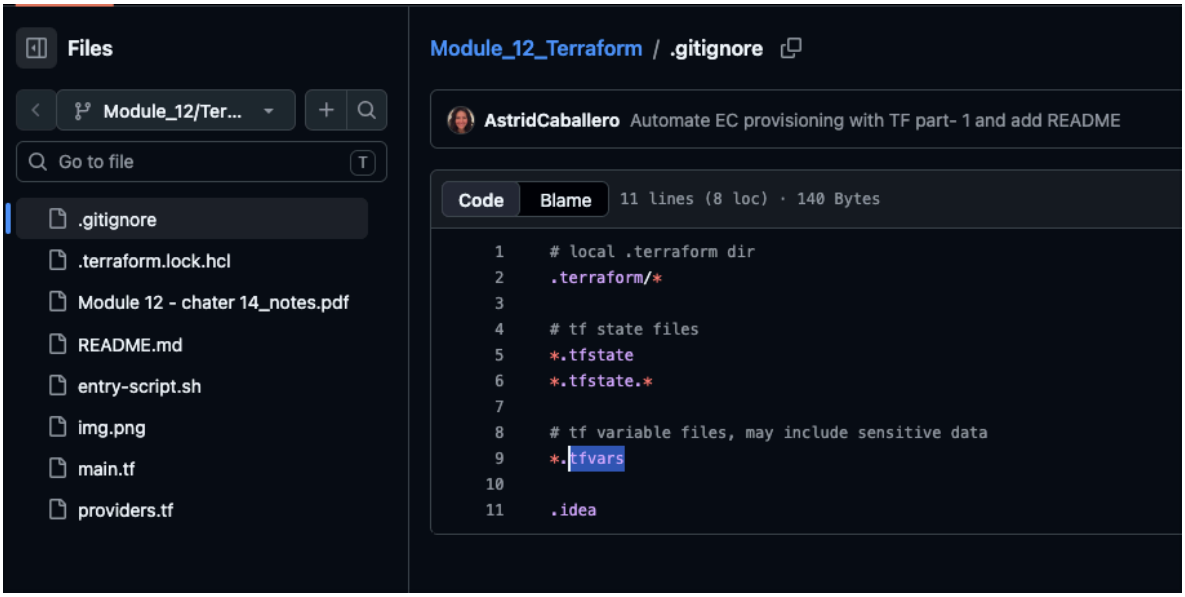
That's precisely why the Dockerfile approach is better — the install is baked into the image, so every time the container starts it's already there.

— end tangent —

So for now I will use part of option 1 from the tangent above but I won't mount docker-compose in the container for this demo



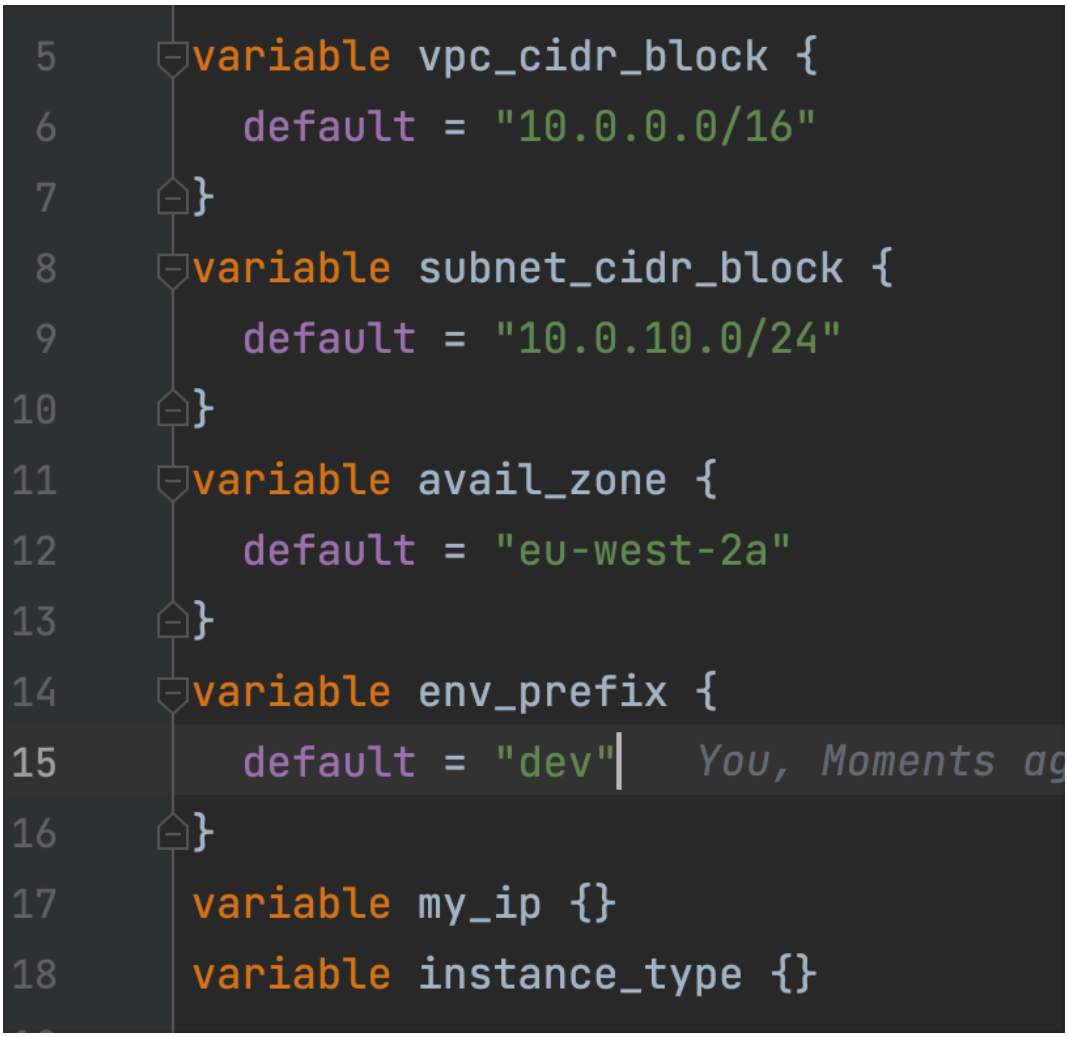
In our previous demos we placed the values of the variables in a file called terraform.tfvars this .tfvars file is in .gitignore hence it was not pushed to the repo as it contained sensitive information



So how do we provide the values to jenkins container environment? How do we set them?

Let's take a look and the variables and remove the public key one because we didn't use it in this demo:

So now we have some variables whose values don't change much so we can give them default values. Remember that default values can be overridden later.



Regarding my ip address, I still want to be able as the human to ssh into my EC@ instance, so I will pass my IP address as the default

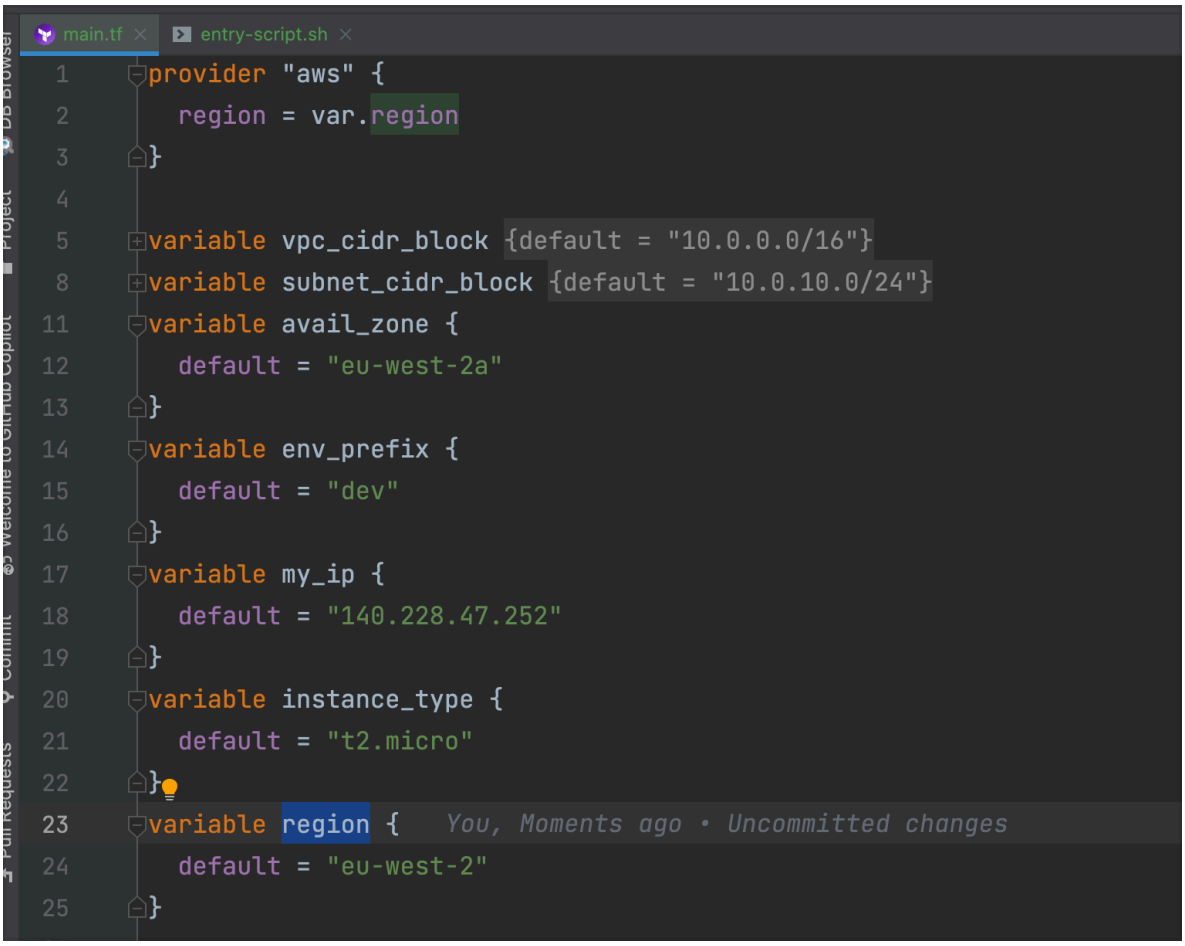
And for the instance type

```
19  }
20  variable instance_type {
21    default = "t2.micro"    You, A mi
22  }
23
```

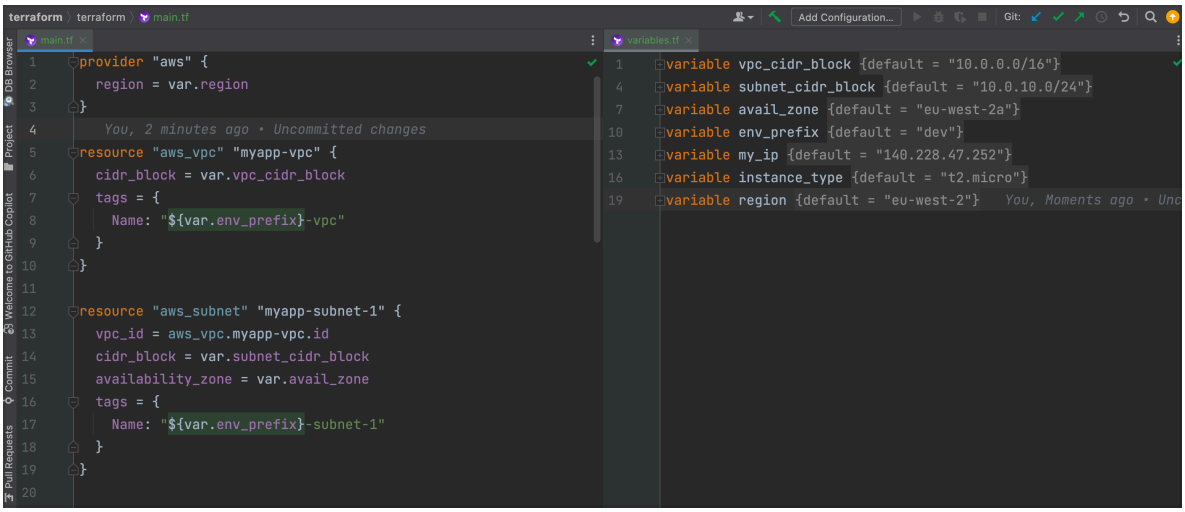
So now with the defaults we don't have to provide the values all the time, only we will provide values that we want to override

We can also parameterise the region

So now I have:



An now I will move all those variables to a new file called ‘variables.tf’



Now the main.tf is cleaner.

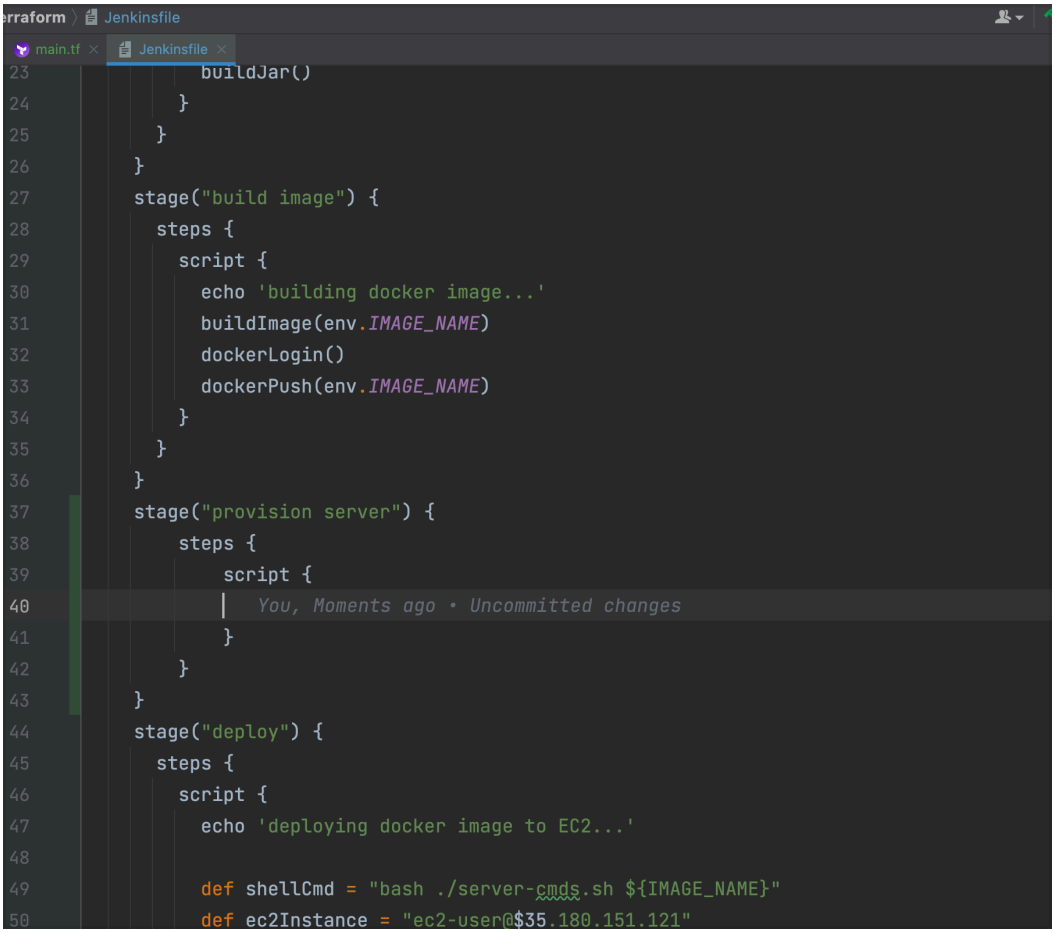
We will keep outputs in the main.tf as we only have one.

```
71  output "ec2_public_ip" {
72    value = aws_instance.myapp-server.public_ip
73  }   You, Moments ago • Uncommitted changes
74
```

Provision stage in Jenkinsfile

The final part is to adjust jenkinsfile for the CI/CD pipeline.

So let's add the stage "provision server" (line 37)



And inside the script we will execute Terraform commands.

And we need to execute the commands from inside the directory where the terraform configuration is which is the terraform folder and for that we use ‘dir’

Now that we are inside the folder ‘terraform’ we can execute the terraform commands
-> terraform init

```
37      stage("provision server") {
38          steps {
39              script {
40                  dir('terraform') {
41                      sh "terraform init"    You, 
42                  }
43              }
44          }
45      }
```

-> terraform apply —auto-approve

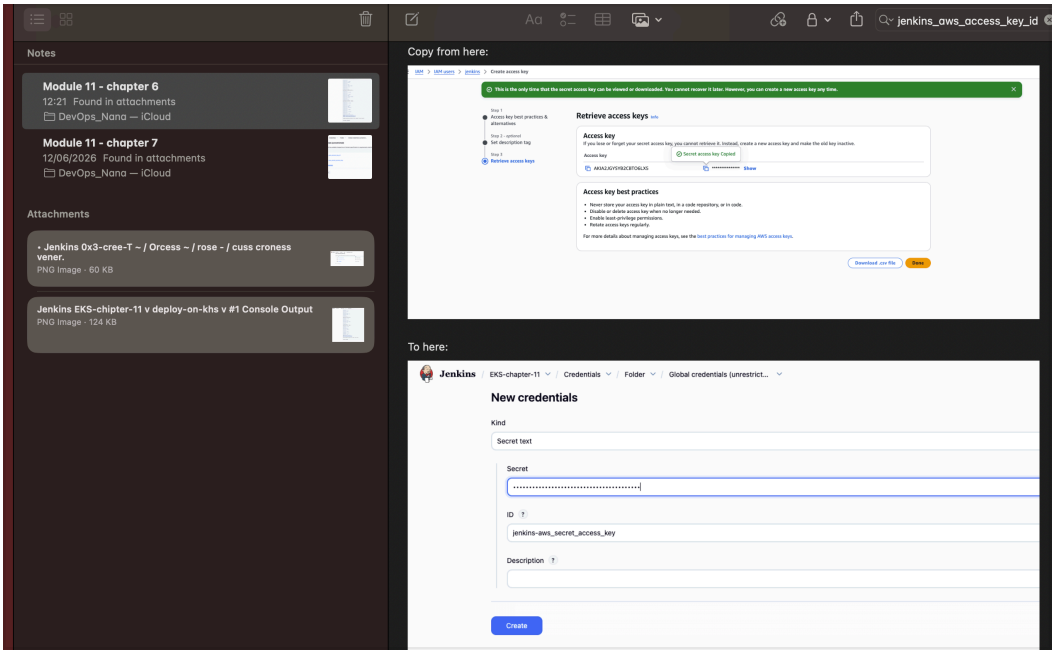
```
36      }
37      stage("provision server") {
38          steps {
39              script {
40                  dir('terraform') {
41                      sh "terraform init"
42                      sh "terraform apply -auto-approve"    You, Momen
43                  }
44              }
45          }
46      }
```

But for terraform ‘apply’ to work we need terraform and Jenkins Server need to authenticate with AWS because we are creating resources in AWS so we need AWS to let us in first.

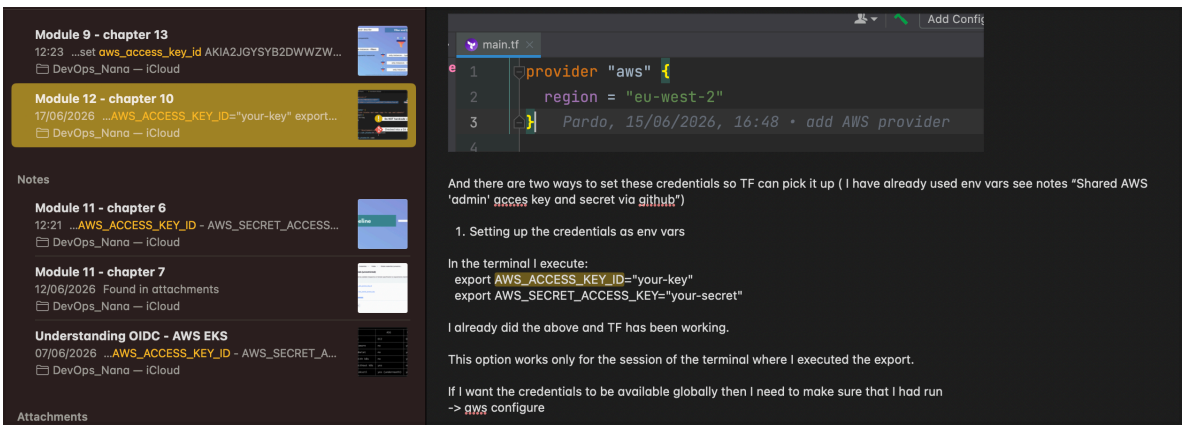
In terraform before we set the credentials inside the ‘provider’ block. The credentials I used were created in AWS IAM for user ‘admin’

```
provider "aws" {
  region = "eu-west-2"
  access_key = "AKIA2JG
  secret_key = "NicXYQo
}
```

The credentials we want to use are the ones I created in AWS IAM for user ‘jenkins’ (I recently change credentials but for user ‘admin’ because by mistake I push them into GitHub)



And then we set them in our laptop as env variables and also we used aws configure (see Module 12 - chapter 10
Environment Variables in Terraform)



So in jenkinsfile we need to set the env variables so the ‘provider’ block in

terraform (main.tf) can grab those env vars and connect to AWS.

```
37      stage("provision server") {
38          environment {
39              You, Moments ago • Uncommitted changes
40          }
41          steps {
42              script {
43                  dir('terraform') {
44                      sh "terraform init"
45                      sh "terraform apply -auto-approve"
46                  }
47              }
48          }
49      }
```

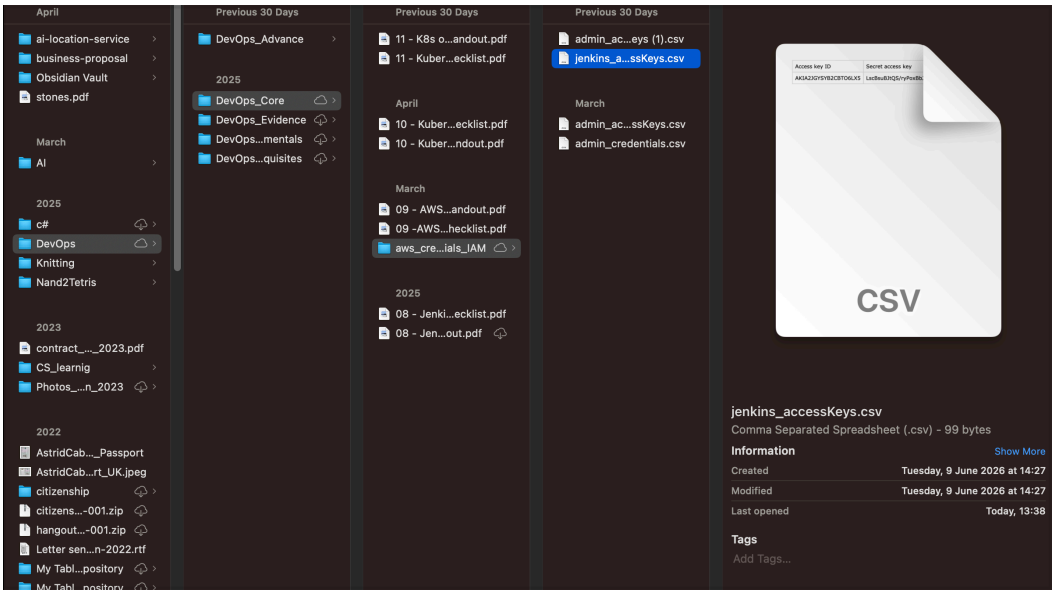
And we provide the two variables

```
37      stage("provision server") {
38          environment {
39              AWS_ACCESS_KEY_ID =
40              AWS_SECRET_ACCESS_KEY =    You, A minute a
41          }
```

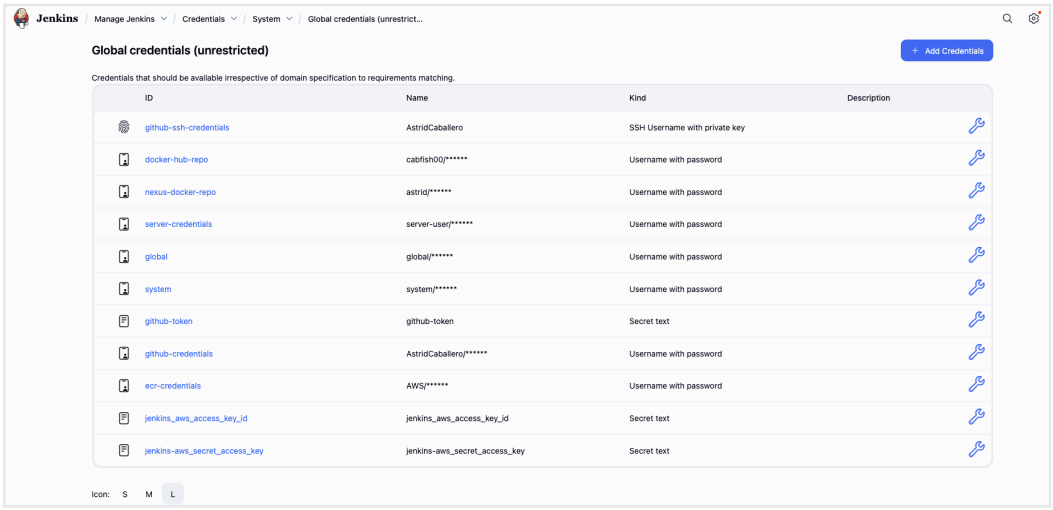
Nana already have the credentials created in jenkins when we worked in the EKS demos. As I mentioned before I created new credentials for AWS IAM user 'jenkins' so I need to update the current credentials I have

But as we can see the current credentials I have are part of the EKS-chapter-11 scope, they are not global credentials like Nana's. So I need to create them again but as global

First I find the file with the AWS credentials for 'Jenkins' user



Then I create them in jenkins



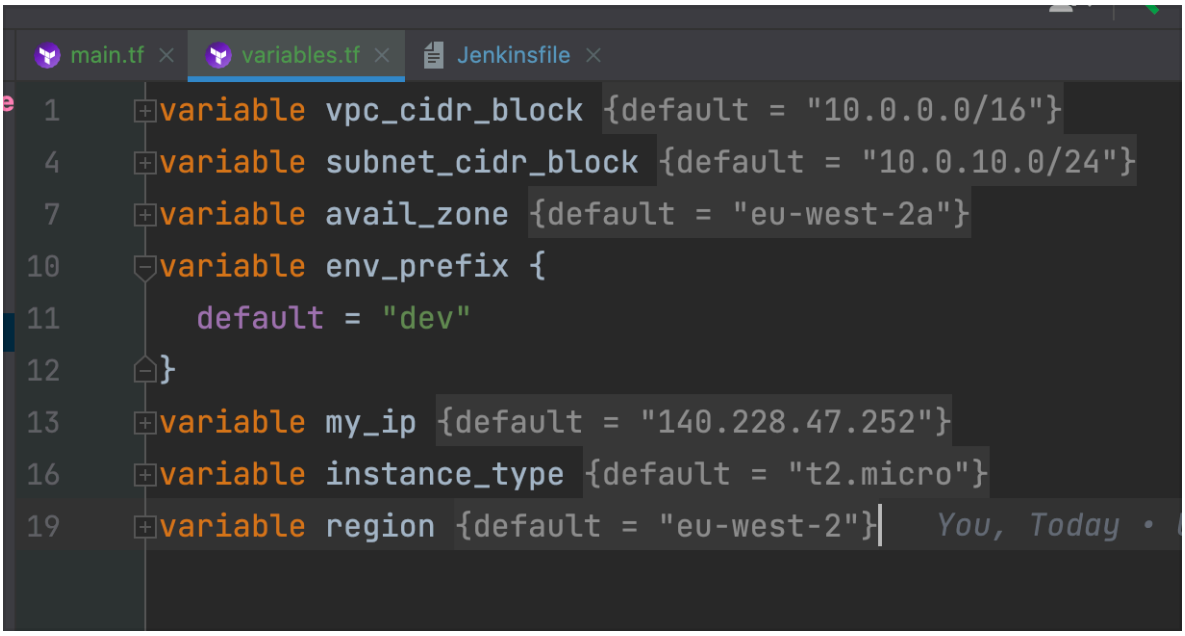
Note: I created these credentials before but locally for EKS-chapter-11 pipeline, I won't delete them (to match the notes rated to that pipeline) although I should do it to avoid confusion as the ids are the same and the secrets are the same so I only need the global credentials.

And now I update the jenkinsfile

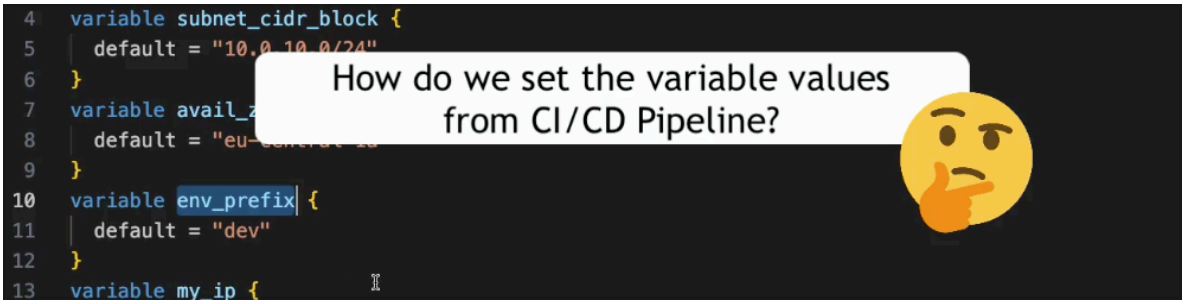
```
35        }
36    }
37    stage("provision server") {
38        environment {   You, Moments ago • Uncommitted changes
39            AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
40            AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secret_access_key')
41        }
42        steps {
```

Now jenkins can get authenticated in AWS and execute the terraform commands 🥳

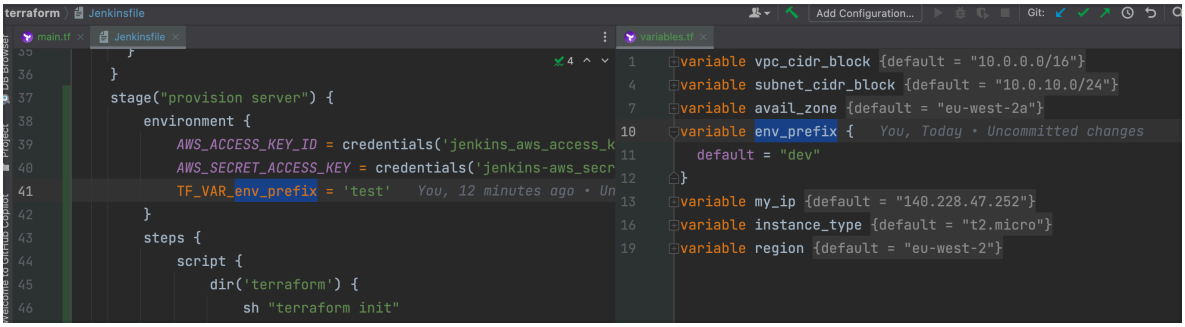
Now we can override variables! Let's say we want to create the infra for a 'test' env instead of 'dev' env (default)



So our CI/CD pipeline is for 'test'



One way is using terraform environment variables -> TF_VAR_<variable_name>



Deploy Stage in Jenkinsfile

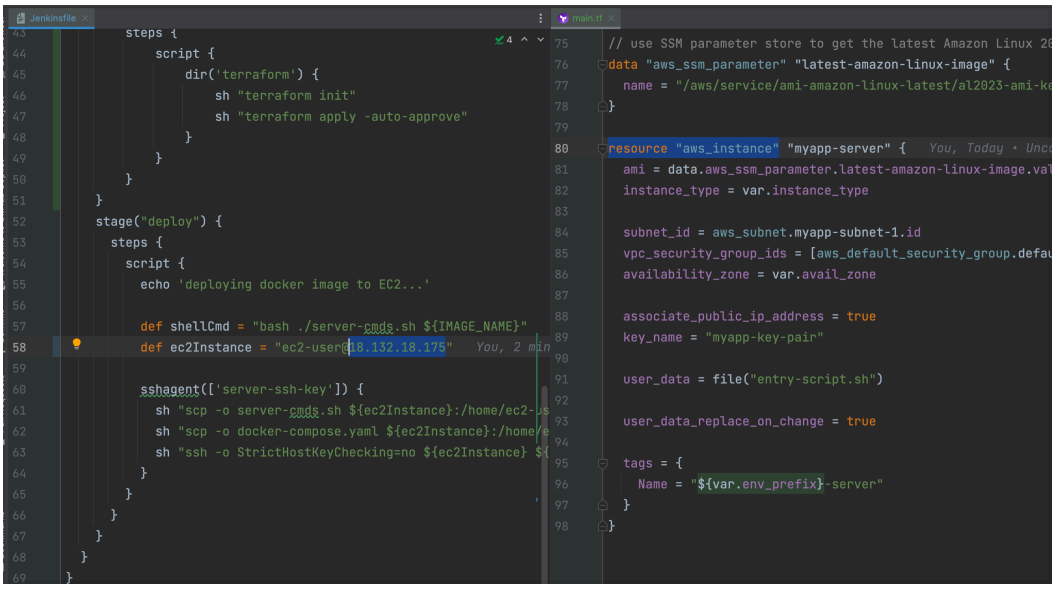
We need to make some adjustments in the 'deploy' stage

First, currently we are hardcoding the ip address of the server

```
52      stage("deploy") {
53        steps {
54          script {
55            echo 'deploying docker image to EC2...'
56
57            def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
58            def ec2Instance = "ec2-user@18.132.18.175"   You, 5 minutes ago • Uncommitt
59
60            sshagent(['server-ssh-key']) {
61              sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
62              sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
63              sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
64            }
65          }
66        }
```

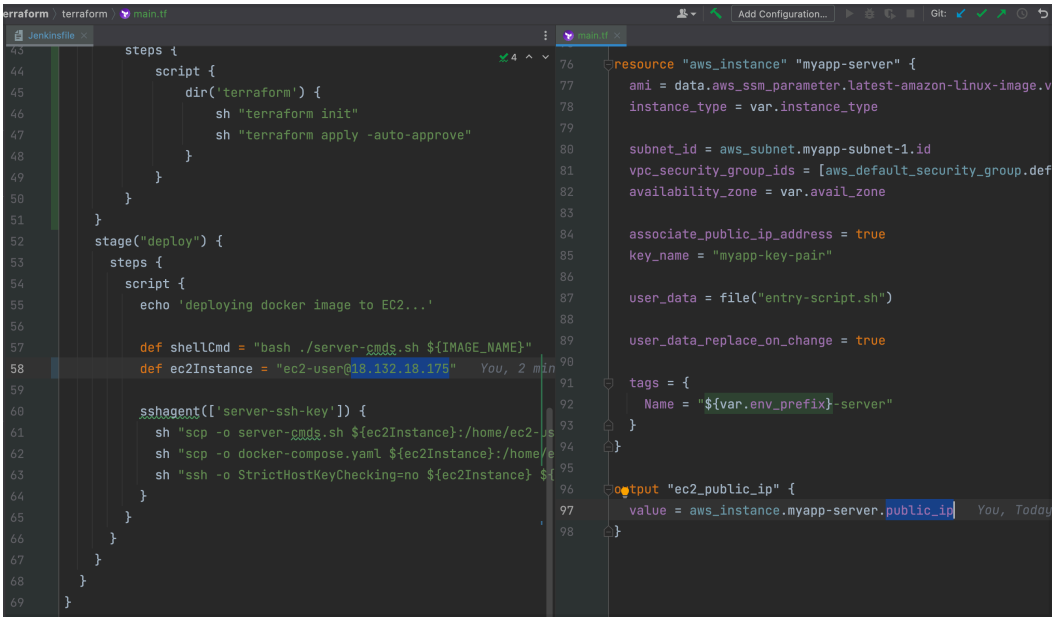
But we are provisioning the server with TF so we don't know the IP address that the EC2 insatnce will have.

So TF will create an EC2 aws_instance resource and we need to pass it to jenkinsfile



How do we pass the resource?

TF has the resource output and we already have an output that returns the public_ip of the server



```
steps {
    script {
        dir('terraform') {
            sh "terraform init"
            sh "terraform apply -auto-approve"
        }
    }
}
}
stage("deploy") {
steps {
  script {
    echo 'deploying docker image to EC2...'

    def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
    def ec2Instance = "ec2-user@18.132.18.175"   You, 2 min

    sshagent(['server-ssh-key']) {
      sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-us
      sh "scp -o docker-compose.yaml ${ec2Instance}:/home/e
      sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${
    }
  }
}
}
}
}
```

```
resource "aws_instance" "myapp-server" {
  ami = data.aws_ssm_parameter.latest-amazon-linux-image.v
  instance_type = var.instance_type

  subnet_id = aws_subnet.myapp-subnet-1.id
  vpc_security_group_ids = [aws_default_security_group.def
  availability_zone = var.avail_zone

  associate_public_ip_address = true
  key_name = "myapp-key-pair"

  user_data = file("entry-script.sh")

  user_data_replace_on_change = true

  tags = {
    Name = "${var.env_prefix}-server"
  }
}

output "ec2_public_ip" {
  value = aws_instance.myapp-server.public_ip   You, Today
}
```

So we can get the ‘provision server’ stage to inform jenkins of the output specifically ‘ec2_public_ip’

```
}
stage("provision server") {
    environment {
        AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_k
        AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secr
        TF_VAR_env_prefix = 'test'
    }
    steps {
        script {
            dir('terraform') {
                sh "terraform init"
                sh "terraform apply -auto-approve"
                sh "terraform output ec2_public_ip"   You, 20
            }
        }
    }
}
stage("deploy") {
```

```
vpc_security_group_ids = [aws_default_security_group.defa
availability_zone = var.avail_zone

associate_public_ip_address = true
key_name = "myapp-key-pair"

user_data = file("entry-script.sh")

user_data_replace_on_change = true

tags = {
  Name = "${var.env_prefix}-server"
}
}

output "ec2_public_ip" {   You, 14 minutes ago • Uncommitte
  value = aws_instance.myapp-server.public_ip
}
```

By assigning the result of the -> sh "terraform output ec2_public_ip" to a env var that Jenkins can later access:

```
steps {
    script {
        dir('terraform') {
            sh "terraform init"
            sh "terraform apply -auto-approve"
            EC2_PUBLIC_IP = sh "terraform output ec2_public_ip"   You, 3 minutes a
        }
    }
}
}
```

But the above doesn’t really work, first we need to print out the sh result to the standard output in order to save it to a variable in this case an env var.

Breaking it down:

- sh(...) — runs a shell command inside the pipeline
- script: "terraform output ec2_public_ip" — the command to run; asks Terraform for the value of the ec2_public_ip output variable
- returnStdout: true — instead of just running the command, capture what it prints to stdout and return it as a string. Without this, sh just returns the exit code (0 for success)
- .trim() — removes any trailing newline/whitespace that the shell command adds at the end
- EC2_PUBLIC_IP = — stores the result in a pipeline variable so you can use it later (e.g. to SSH into the instance)

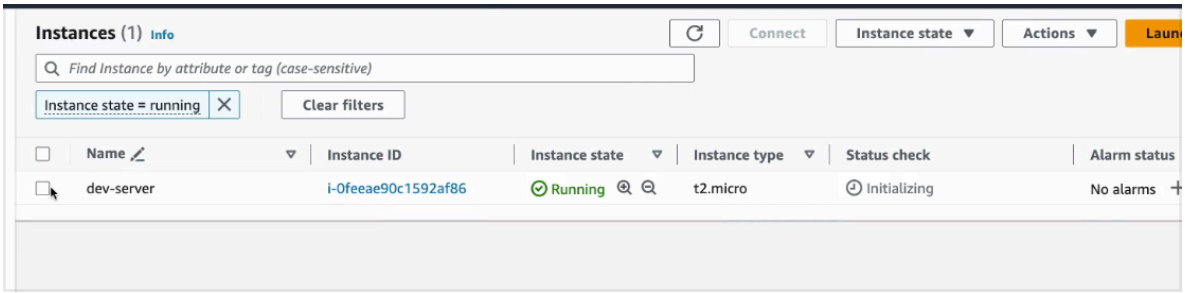
So in plain English: run terraform output ec2_public_ip, grab what it prints, strip the whitespace, and store it in EC2_PUBLIC_IP.

Now we can use EC2_PUBLIC_IP in the ‘deploy’ stage

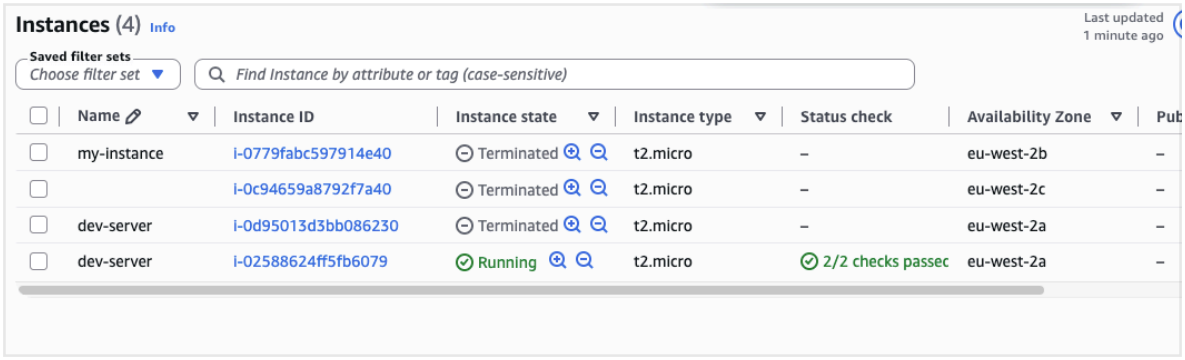
```
Jenkinsfile
43          steps {
44              script {
45                  dir('terraform') {
46                      sh "terraform init"
47                      sh "terraform apply -auto-approve"
48                      EC2_PUBLIC_IP = sh(
49                          script: "terraform output ec2_public_ip",
50                          returnStdout: true
51                      ).trim()
52                  }
53              }
54          }
55      }
56      stage("deploy") {
57        steps {
58          script {
59            echo 'deploying docker image to EC2...'
60
61            def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
62            def ec2Instance = "ec2-user@${EC2_PUBLIC_IP}"    You, 2 minutes ago • Uncommitted
63
64            sshagent(['server-ssh-key']) {
65              sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
66              sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
67              sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
68            }
69          }
70        }
```

From previous demos we noticed that after creating the EC2 instance using TF it took some time for the EC2 to be ready (up and running) it takes time to initialised.

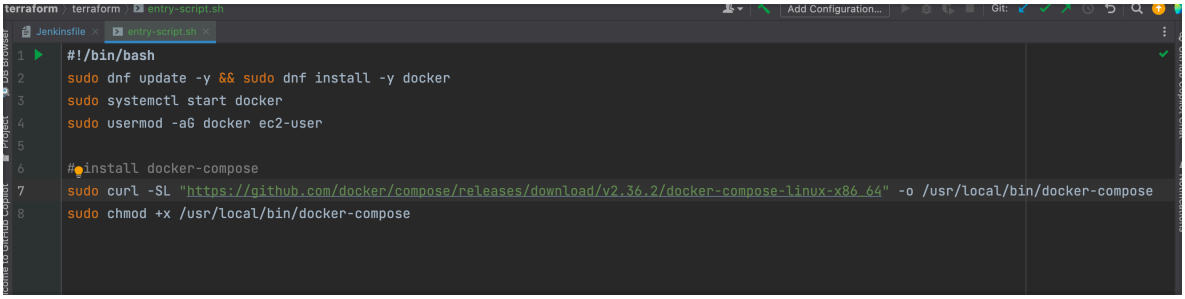
We could see that in the AWS UI, where the ‘instance state’ was ‘running’ but the ‘status check’ was ‘initializing’



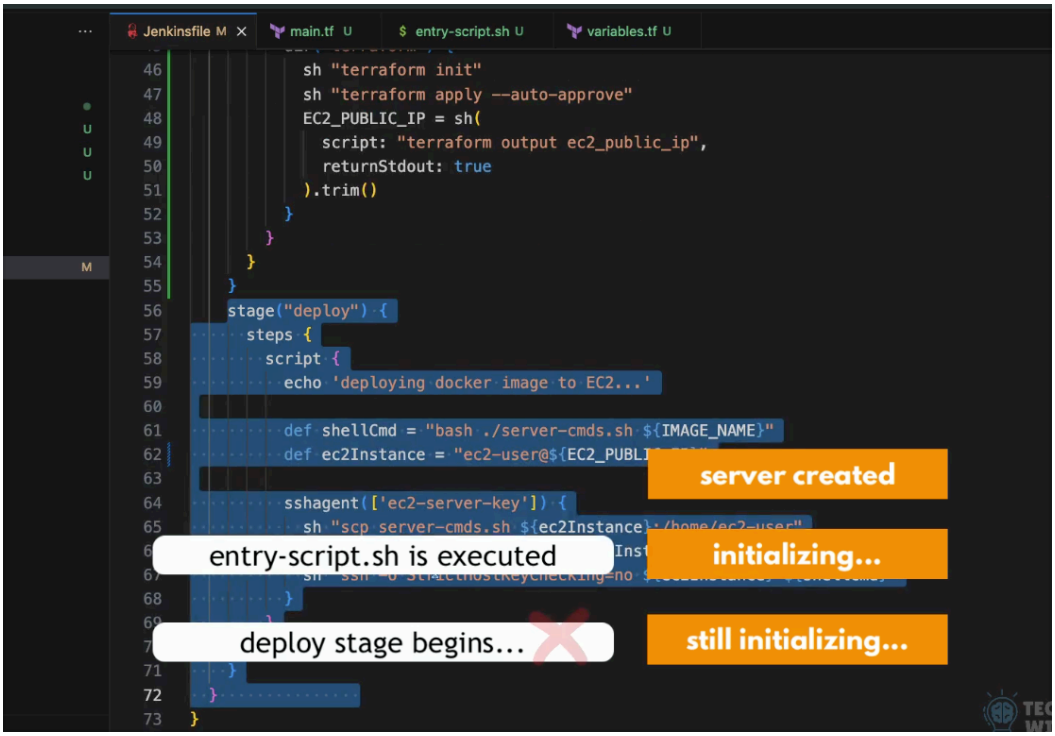
And once the initialisation is done then we see the green check marks displayed (2/2 check passed) like we see in this other example:



And part of the initialisation is to run the script

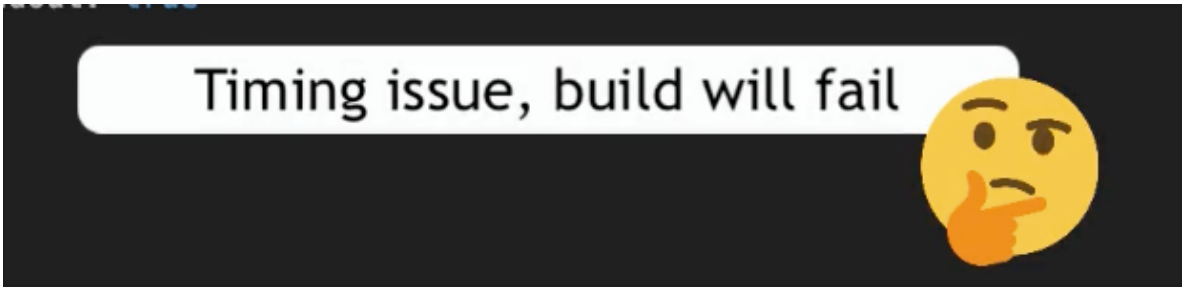


The thing is that as far as ‘provision server’ stage is concerned, TF has finished its work. AWS is taking time to finish the initialization. So jenkins file now moves to ‘deploy’ stage but in reality the server is not ready yet for deployment, it is still initialising.



So when jenkins tries to execute the commands of the ‘deploy’ stage it fails

This is a **timing** issue ⏱️



This will happen the first time jenkins runs this pipeline as it will have to create the server but next time jenkins runs the pipeline the server already exists so it won't be an issue.

Two options:

- Execute 'provision server' stage as the first stage
- Add a sleep to the 'deploy' stage before executing the commands to give some time for the initialisation of the server to be done.

So let's see the sleep code:

```
55      }
56      stage("deploy") {
57        steps {
58          script {
59            sleep(time: 90, unit: "SECONDS")   You, Moments ago • Uncommitted changes
60            echo 'deploying docker image to EC2...'
61
62            def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
63            def ec2Instance = "ec2-user@${EC2_PUBLIC_IP}"
64
65            sshagent(['server-ssh-key']) {
66              sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
67              sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
68              sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
69            }
70          }
```

And we can add a couple of echo to log that we are waiting and the ec2 public ip see lines 59 and 63:

```
57          steps {
58            script {
59              echo "waiting for EC2 server to initialize"
60              sleep(time: 90, unit: "SECONDS")
61
62              echo 'deploying docker image to EC2...'
63              echo "${EC2_PUBLIC_IP}"
```

The above solves the timing issue but the code needs to be optimised. Once the server exists when we run the pipeline it will always wait for a minute and a half for no reason.

So to improve this we can wrap the sleep code in an IF statement that checks whether the Ec2 instance initialisation is done or not to decide whether to sleep or not.

Another adjustment is to add ‘StrictHostKeyChecking=no’ To all of the secure copies -> scp commands

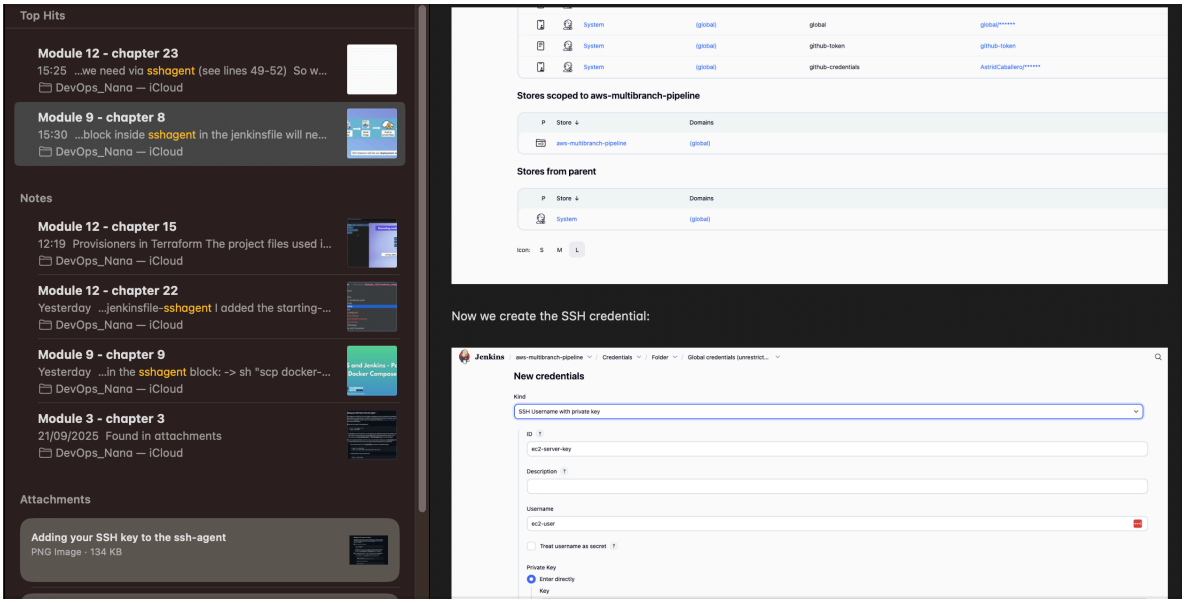
To turn off the strict host key checking when we are connecting to the EC2 instance

```
67
68      sshagent(['server-ssh-key']) {
69        sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
70        sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
71        sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"   Janashia, 15/10/2
72      }
73    }
74  }
```

So now our file looks like this

```
67
68    sshagent(['server-ssh-key']) {
69      sh "scp -o StrictHostKeyChecking=no server-cmds.sh ${ec2Instance}:/home/ec2-user"
70      sh "scp -o StrictHostKeyChecking=no docker-compose.yaml ${ec2Instance}:/home/ec2-user"
71      sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
72    }
```

We created sshagent and the ‘ec2-server-key’ in Module 9 - chapter 8



‘server-ssh-key’ is the one we created at the beginning in my ‘terraform-pipeline’